Chapter 2
Overview of the Projects

Our general plan is to craft analytic, decision support modules and applications. These applications support decision-making by providing summaries of available data to the stakeholders. Decision-making spans a spectrum from uncovering new relationships among variables to confirming that data variation is random noise within narrow limits. The processing will start with acquiring data and moving it through several stages until statistical summaries can be presented.

The processing will be decomposed into several stages. Each stage will be built as a core concept application. There will be subsequent projects to add features to the core application. In some cases, a number of features will be added to several projects all combined into a single chapter.

The stages are inspired by the **Extract-Transform-Load** (**ETL**) architectural pattern. The design in this book expands on the ETL design with a number of additional steps. The words have been changed because the legacy terminology can be misleading. These features – often required for real-world pragmatic applications — will be inserted as additional stages in a pipeline.

Once the data is cleaned and standardized, then the book will describe some simple statistical models. The analysis will stop there. You are urged to move to more advanced books that cover AI and machine learning.

There are 22 distinct projects, many of which build on previous results. It's not required to do all of the projects in order. When skipping a project, however, it's important to read the description and deliverables for the project being skipped. This can help to more fully understand the context for the later projects.

This chapter will cover our overall architectural approach to creating a complete sequence of data analysis programs. We'll use the following multi-stage approach:

- Data acquisition
- Inspection of data
- Cleaning data; this includes validating, converting, standardizing, and saving intermediate results
- Summarizing, and modeling data
- Creating more sophisticated statistical models

The stages fit together as shown in ***Figure 2.1***.

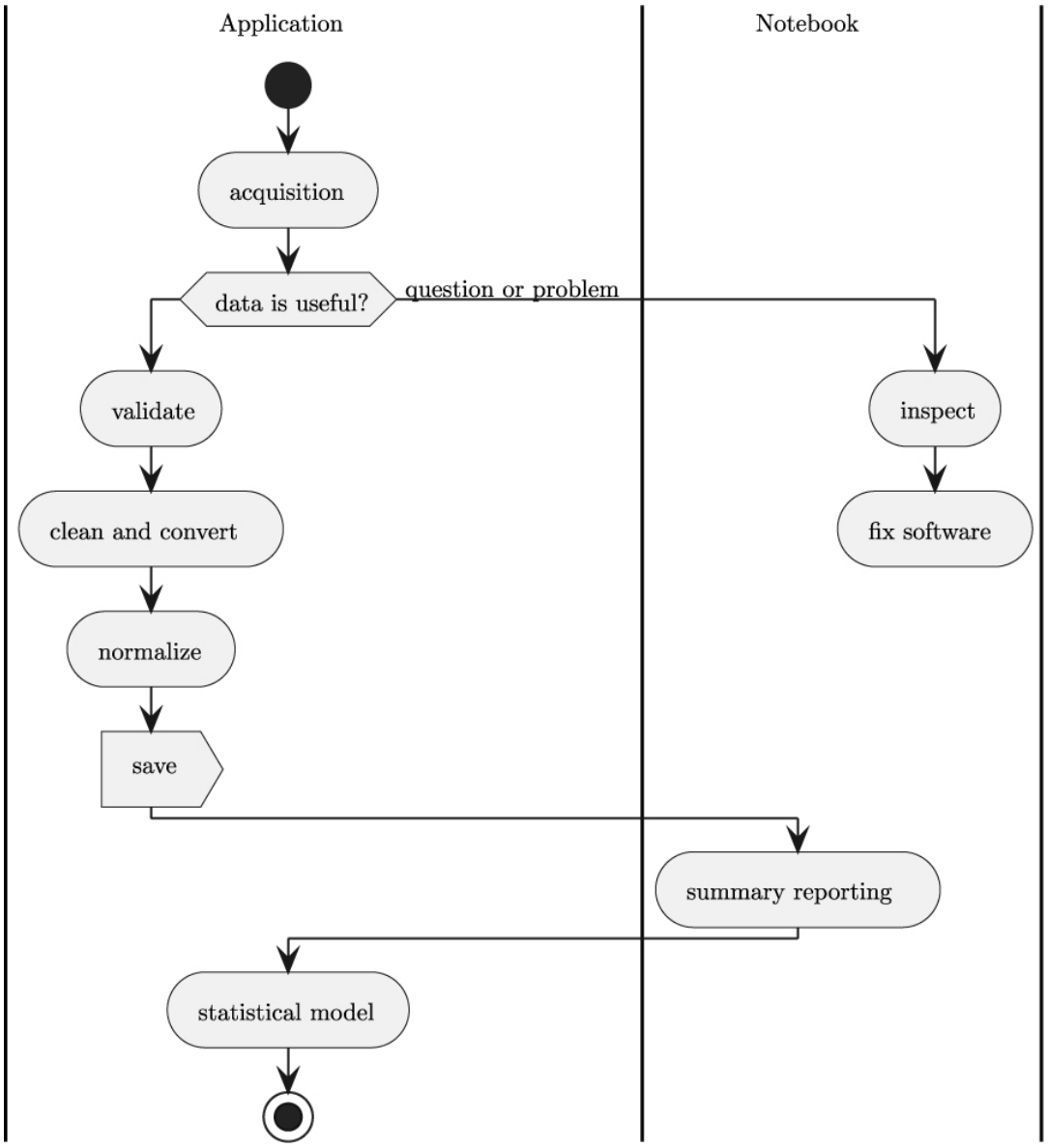

Figure 2.1: Data Analysis Pipeline

A central idea behind this is *separation of concerns*. Each stage is a distinct operation, and each stage may evolve independently of the others. For example, there may be multiple sources of data, leading to several distinct data acquisition implementations, each of which creates a common internal representation to permit a single, uniform inspection tool.

Similarly, data cleansing problems seem to arise almost randomly in organizations, leading to a need to add distinct validation and standardization operations. The idea is to allocate responsibility for semantic special cases and exceptions in this stage of the pipeline.

One of the architectural ideas is to mix automated applications and a few manual JupyterLab notebooks into an integrated whole. The notebooks are essential for troubleshooting questions or problems. For elegant reports and presentations, notebooks are also very useful. While Python applications can produce tidy PDF files with polished reporting, it seems a bit easier to publish a notebook with analysis and findings.

We'll start with the acquisition stage of processing.

2.1 General data acquisition

All data analysis processing starts with the essential step of acquiring the data from a source.

The above statement seems almost silly, but failures in this effort often lead to complicated rework later. It's essential to recognize that data exists in these two essential forms:

- Python objects, usable in analytic programs. While the obvious candidates are numbers and strings, this includes using packages like **Pillow** to operate on images as Python objects. A package like **librosa** can create objects representing audio data.
- A serialization of a Python object. There are many choices here:
  - Text. Some kind of string. There are numerous syntax variants, including CSV, JSON, TOML, YAML, HTML, XML, etc.
  - Pickled Python Objects. These are created by the `pickle` module.
  - Binary Formats. Tools like Protobuf can serialize native Python objects into a stream of bytes. Some YAML extensions, similarly, can serialize an object in a binary format that isn't text. Images and audio samples are often stored in compressed binary formats.

The format for the source data is — almost universally — not fixed by any rules or conventions. Writing an application based on the assumption that source data is **always** a CSV-format file can lead to problems when a new format is required.

It's best to treat all input formats as subject to change. The data — once acquired — can be saved in a common format used by the analysis pipeline, and independent of the source format (we'll get to the persistence in ***Clean, validate, standardize, and persist***).

We'll start with Project 1.1: "Acquire Data". This will build the Data Acquisition Base Application. It will acquire CSV-format data and serve as the basis for adding formats in later projects.

There are a number of variants on how data is acquired. In the next few chapters, we'll look at some alternative data extraction approaches.

2.2 Acquisition via Extract

Since data formats are in a constant state of flux, it's helpful to understand how to add and modify data formats. These projects will all build on Project 1.1 by adding features to the base application. The following projects are designed around alternative sources for data:

- Project 1.2: "Acquire Web Data from an API". This project will acquire data from web services using JSON format.
- Project 1.3: "Acquire Web Data from HTML". This project will acquire data from a web page by scraping the HTML.
- Two separate projects are part of gathering data from a SQL database:
  - Project 1.4: "Build a Local Database". This is a necessary sidebar project to build a local SQL database. This is necessary because SQL databases accessible by the public are a rarity. It's more secure to build our own demonstration database.

- Project 1.5: ”Acquire Data from a Local Database”. Once a database is available, we can acquire data from a SQL extract.

These projects will focus on data represented as text. For CSV files, the data is text; an application must convert it to a more useful Python type. HTML pages, also, are pure text. Sometimes, additional attributes are provided to suggest the text should be treated as a number. A SQL database is often populated with non-text data. To be consistent, the SQL data should be serialized as text. The acquisition applications all share a common approach of working with text.

These applications will also minimize the transformations applied to the source data. To process the data consistently, it’s helpful to make a shift to a common format. As we’ll see in ***Chapter 3***, ***Project 1.1: Data Acquisition Base Application*** the NDJSON format provides a useful structure that can often be mapped back to source files.

After acquiring new data, it’s prudent to do a manual inspection. This is often done a few times at the start of application development. After that, inspection is only done to diagnose problems with the source data. The next few chapters will cover projects to inspect data.

2.3 Inspection

Data inspection needs to be done when starting development. It’s essential to survey new data to be sure it really is what’s needed to solve the user’s problems. A common frustration is incomplete or inconsistent data, and these problems need to be exposed as soon as possible to avoid wasting time and effort creating software to process data that doesn’t really exist.

Additionally, data is inspected manually to uncover problems. It’s important to recognize that data sources are in a constant state of flux. As applications evolve and mature, the data provided for analysis will change. In many cases, data analytics applications discover other en-

terprise changes after the fact via invalid data. It's important to understand the evolution via good data inspection tools.

Inspection is an inherently manual process. Therefore, we're going to use JupyterLab to create notebooks to look at the data and determine some basic features.

In rare cases where privacy is important, developers may not be allowed to do data inspection. More privileged people — with permission to see payment card or healthcare details — may be part of data inspection. This means an inspection notebook may be something created by a developer for use by stakeholders.

In many cases, a data inspection notebook can be the start of a fully-automated data cleansing application. A developer can extract notebook cells as functions, building a module that's usable from both notebook and application. The cell results can be used to create unit test cases.

The stage in the pipeline requires a number of inspection projects:

- Project 2.1: "Inspect Data". This will build a core data inspection notebook with enough features to confirm that some of the acquired data is likely to be valid.
- Project 2.2: "Inspect Data: Cardinal Domains". This project will add analysis features for measurements, dates, and times. These are cardinal domains that reflect measures and counts.
- Project 2.3: "Inspect Data: Nominal and Ordinary Domains". This project will add analysis features for text or coded numeric data. This includes nominal data and ordinal numeric domains. It's important to recognize that US Zip Codes are digit strings, not numbers.
- Project 2.4: "Inspect Data: Reference Data". This notebook will include features to find reference domains when working with data

that has been normalized and decomposed into subsets with references via coded ”key” values.

- Project 2.5: ”Define a Reusable Schema”. As a final step, it can help define a formal schema, and related metadata, using the JSON Schema standard.

While some of these projects seem to be one-time efforts, they often need to be written with some care. In many cases, a notebook will need to be reused when there’s a problem. It helps to provide adequate explanations and test cases to help refresh someone’s memory on details of the data and what are known problem areas. Additionally, notebooks may serve as examples for test cases and the design of Python classes or functions to automate cleaning, validating, or standardizing data.

After a detailed inspection, we can then build applications to automate cleaning, validating, and normalizing the values. The next batch of projects will address this stage of the pipeline.

2.4 Clean, validate, standardize, and persist

Once the data is understood in a general sense, it makes sense to write applications to clean up any serialization problems, and perform more formal tests to be sure the data really is valid. One frustratingly common problem is receiving duplicate files of data; this can happen when scheduled processing was disrupted somewhere else in the enterprise, and a previous period’s files were reused for analysis.

The validation testing is sometimes part of cleaning. If the data contains any unexpected invalid values, it may be necessary to reject it. In other cases, known problems can be resolved as part of analytics by replacing invalid data with valid data. An example of this is US Postal Codes, which are (sometimes) translated into numbers, and the leading zeros are lost.

These stages in the data analysis pipeline are described by a number of projects:

- Project 3.1: ”Clean Data”. This builds the data cleaning base application. The design details can come from the data inspection notebooks.
- Project 3.2: ”Clean and Validate”. These features will validate and convert numeric fields.
- Project 3.3: ”Clean and Validate Text and Codes”. The validation of text fields and numeric coded fields requires somewhat more complex designs.
- Project 3.4: ”Clean and Validate References”. When data arrives from separate sources, it is essential to validate references among those sources.
- Project 3.5: ”Standardize Data”. Some data sources require standardizing to create common codes and ranges.
- Project 3.6: ”Acquire and Clean Pipeline”. It’s often helpful to integrate the acquisition, cleaning, validating, and standardizing into a single pipeline.
- Project 3.7: ”Acquire, Clean, and Save”. One key architectural feature of this pipeline is saving intermediate files in a common format, distinct from the data sources.
- Project 3.8: ”Data Provider Web Service”. In many enterprises, an internal web service and API are expected as sources for analytic data. This project will wrap the data acquisition pipeline into a RESTful web service.

In these projects, we’ll transform the text values from the acquisition applications into more useful Python objects like integers, floating-point values, decimal values, and date-time values.

Once the data is cleaned and validated, the exploration can continue. The first step is to summarize the data, again, using a Jupyter notebook to create readable, publishable reports and presentations. The next chapters will explore the work of summarizing data.

2.5 Summarize and analyze

Summarizing data in a useful form is more art than technology. It can be difficult to know how best to present information to people to help them make more valuable, or helpful decisions.

There are a few projects to capture the essence of summaries and initial analysis:

- Project 4.1: ”A Data Dashboard”. This notebook will show a number of visual analysis techniques.
- Project 4.2: ”A Published Report”. A notebook can be saved as a PDF file, creating a report that’s easily shared.

The initial work of summarizing and creating shared, published reports sets the stage for more formal, automated reporting. The next set of projects builds modules that provide deeper and more sophisticated statistical models.

2.6 Statistical modeling

The point of data analysis is to digest raw data and present information to people to support their decision-making. The previous stages of the pipeline have prepared two important things:

- Raw data has been cleaned and standardized to provide data that are relatively easy to analyze.
- The process of inspecting and summarizing the data has helped analysts, developers, and, ultimately, users understand what the information means.

The confluence of data and deeper meaning creates significant value for an enterprise. The analysis process can continue as more formalized statistical modeling. This, in turn, may lead to artificial intelligence (AI) and machine learning (ML) applications.

The processing pipeline includes these projects to gather summaries of individual variables as well as combinations of variables:

- Project 5.1: ”Statistical Model: Core Processing”. This project builds the base application for applying statistical models and saving parameters about the data. This will focus on summaries like mean, median, mode, and variance.
- Project 5.2: ”Statistical Model: Relationships”. It’s common to want to know the relationships among variables. This includes measures like correlation among variables.

This sequence of stages produces high-quality data and provides ways to diagnose and debug problems with data sources. The sequence of projects will illustrate how automated solutions and interactive inspection can be used to create useful, timely, insightful reporting and analysis.

2.7 Data contracts

We will touch on data contracts at various stages in this pipeline. This application’s data acquisition, for example, may have a formalized contract with a data provider. It’s also possible that an informal data contract, in the form of a schema definition, or an API is all that’s available.

In ***Chapter 8***, ***Project 2.5: Schema and Metadata*** we’ll consider some schema publication concerns. In ***Chapter 11***, ***Project 3.7: Interim Data Persistence*** we’ll consider the schema provided to downstream applications. These two topics are related to a formal data contract, but this book won’t delve deeply into data contracts, how they’re created, or how they might be used.

2.8 Summary

This data analysis pipeline moves data from sources through a series of stages to create clean, valid, standardized data. The general flow supports a variety of needs and permits a great deal of customization and extension.

For developers with an interest in data science or machine learning, these projects cover what is sometimes called the ”data wrangling” part of data science or machine learning. It can be a significant complication as data is understood and differences among data sources are resolved and explored. These are the — sometimes difficult — preparatory steps prior to building a model that can be used for AI decision-making.

For readers with an interest in the web, this kind of data processing and extraction is part of presenting data via a web application API or website. Project 3.7 creates a web server, and will be of particular interest. Because the web service requires clean data, the preceding projects are helpful for creating data that can be published.

For folks with an automation or IoT interest, *Part 2* explains how to use Jupyter Notebooks to gather and inspect data. This is a common need, and the various steps to clean, validate, and standardize data become all the more important when dealing with real-world devices subject to the vagaries of temperature and voltage.

We’ve looked at the following multi-stage approach to doing data analysis:

- Data Acquisition
- Inspection of Data
- Clean, Validate, Standardize, and Persist
- Summarize and Analyze
- Create a Statistical Model

This pipeline follows the Extract-Transform-Load (ETL) concept. The terms have been changed because the legacy words are sometimes misleading. Our acquisition stage overlaps with what is understood as the ”Extract” operation. For some developers, Extract is limited to database extracts; we’d like to go beyond that to include other data source transformations. Our cleaning, validating, and standardizing stages are usually combined into the ”Transform” operation. Saving the clean data is generally the objective of ”Load”; we’re not emphasizing a database load, but instead, we’ll use files.

Throughout the book, we’ll describe each project’s objective and provide the foundation of a sound technical approach. The details of the implementation are up to you. We’ll enumerate the deliverables; this may repeat some of the information from ***Chapter 1***, ***Project Zero: A Template for Other Projects***. The book provides a great deal of information on acceptance test cases and unit test cases — the definition of done. By covering the approach, we’ve left room for you to design and implement the needed application software.

In the next chapter, we’ll build the first data acquisition project. This will work with CSV-format files. Later projects will work with database extracts and web services.